EXPT NO: 6	A PYTHON PROGRAM TO SOLVE AN IPP USING THE BRANCH AND BOUND METHOD
DATE:	

AIM:

To solve any integer programming problem using the Branch and Bound method in Python.

PROCEDURE:

STEP 1 :Formulate the integer programming problem.

STEP 2 : Implement the Branch and Bound method in Python

STEP 3 : Solve the problem using the implemented method.

STEP 4 : Verify the solution.

PROBLEM – 1: SOLVE THE FOLLOWING INTEGER PROGRAMMING PROBLEM USING THE BRANCH AND BOUND METHOD

Maximize Z = 3x + 5y Subject to:

- $2x + 4y \leq 25$
- $x \leq 8$
- $2y \leq 10$
- $x, y \geq 0$ and integers

CODE

```
import numpy as np

from scipy.optimize import linprog

from queue import Queue

def solve_lp(c, A, b, bounds):

    return linprog(c, A_ub=A, b_ub=b, bounds=bounds, method='highs')

def branch_and_bound(c, A, b, bounds, maximize=True):

    Q = Queue()

    Q.put((c, A, b, bounds))

    best_solution = None
```

```
    best_value = float('-inf') if maximize else float('inf')
    while not Q.empty():
        current_problem = Q.get()
        res = solve_lp(*current_problem)
        if res.success:
            solution = res.x
            value = -res.fun if maximize else res.fun
            if all(np.isclose(solution, np.round(solution))):
                if (maximize and value > best_value) or (not maximize and value < best_value):
                    best_value = value
                    best_solution = solution
            else:
                for i in range(len(solution)):
                    if not np.isclose(solution[i], np.round(solution[i])):
                        lb = list(current_problem[3])
                        ub = list(current_problem[3])
                        lb[i] = (lb[i][0], np.floor(solution[i]))
                        ub[i] = (np.ceil(solution[i]), ub[i][1])
                        Q.put((current_problem[0], current_problem[1], current_problem[2], lb))
                        Q.put((current_problem[0], current_problem[1], current_problem[2], ub))
                        break
    return np.round(best_solution), best_value
c = [-3, -5]
A = [[2, 4], [1, 0], [0, 2]]
b = [25, 8, 10]
bounds = [(0, None), (0, None)]
solution, value = branch_and_bound(c, A, b, bounds, maximize=True)
```

```
print("Problem 1: Max Z = 3x + 5y")

print("Optimal solution:", solution)

print("Optimal value:", value)
```

OUTPUT

```
Problem 1: Max Z = 3x + 5y

Optimal solution: [8. 2.]

 Optimal value: 34.0
```

PROBLEM – 2: SOLVE THE FOLLOWING INTEGER PROGRAMMING PROBLEM USING THE BRANCH AND BOUND METHOD

Maximize:

$$Z=4x+6yZ=4x+6y$$

Subject to:

$x+2y\leq 14x+2y\leq 143x+y\leq 183x+y\leq 18x,y\geq 0$ *and integers*$x,y\geq 0$ and integers

CODE

```
import numpy as np

from scipy.optimize import linprog

from queue import Queue

def solve_lp(c, A, b, bounds):

    return linprog(c, A_ub=A, b_ub=b, bounds=bounds, method='highs')

def branch_and_bound(c, A, b, bounds, maximize=True):

    Q = Queue()

    Q.put((c, A, b, bounds))

    best_solution = None

    best_value = float('-inf') if maximize else float('inf')

 while not Q.empty():

        current_problem = Q.get()

        res = solve_lp(*current_problem)

        if res.success:
```

```
        solution = res.x
        value = -res.fun if maximize else res.fun
        if all(np.isclose(solution, np.round(solution))):
            if value > best_value:
                best_value = value
                best_solution = solution
        else:
            for i in range(len(solution)):
                if not np.isclose(solution[i], np.round(solution[i])):
                    lb = list(current_problem[3])
                    ub = list(current_problem[3])
                    lb[i] = (lb[i][0], np.floor(solution[i]))
                    ub[i] = (np.ceil(solution[i]), ub[i][1])
                    Q.put((current_problem[0], current_problem[1], current_problem[2], lb))
                    Q.put((current_problem[0], current_problem[1], current_problem[2], ub))
                    break
    return np.round(best_solution), best_value
 c = [-4, -6]
A = [[1, 2], [3, 1]]
b = [14, 18]
bounds = [(0, None), (0, None)]
solution, value = branch_and_bound(c, A, b, bounds, maximize=True)
print("Problem 2: Max Z = 4x + 6y")
print("Optimal solution:", solution)
print("Optimal value:", value)
```

OUTPUT

```
Problem 2: Max Z = 4x + 6y
```

Optimal solution: [2. 6.]

Optimal value: 44.0

RESULT

Thus, solving any Integer Programming Problem using the Branch and Bound method in Python has been implemented and executed successfully.